

DOCUMENTACIÓN Y CONTROL DE VERSIONES: PILARES DEL DESARROLLO MODERNO



Unidad de Trabajo 2: Documentación y Control de Versiones

1. Introducción

- 1.1. Importancia de la documentación en proyectos de software**
- 1.2. Beneficios del control de versiones**
- 1.3. Flujo de trabajo colaborativo y trazabilidad**

2. Documentación en el desarrollo de aplicaciones web

- 2.1. Tipos de documentación (técnica, de usuario, de instalación, etc.)**
- 2.2. Buenas prácticas para la redacción de documentación**
- 2.3. Herramientas para la generación de documentación (Markdown, Sphinx, Doxygen, etc.)**
- 2.4. Integración de la documentación en el ciclo de vida del software**

3. Sistemas de control de versiones

- 3.1. Conceptos básicos: repositorio, commit, rama, fusión, conflicto**
- 3.2. Tipos de sistemas de control de versiones: centralizados vs distribuidos**
- 3.3. Ventajas del uso de sistemas de control de versiones**

4. Instalación y configuración de un sistema de control de versiones

- 4.1. Instalación de Git en diferentes sistemas operativos**
- 4.2. Configuración inicial de Git (nombre, correo, editor, SSH)**
- 4.3. Creación de un repositorio local y remoto**
- 4.4. Conexión con plataformas de alojamiento (GitHub, GitLab, Bitbucket)**

5. Uso básico y avanzado de Git

- 5.1. Comandos esenciales: init, add, commit, status, log**
- 5.2. Trabajo con ramas: branch, checkout, merge, rebase**
- 5.3. Resolución de conflictos**
- 5.4. Trabajo colaborativo con repositorios remotos: clone, fetch, pull, push**
- 5.5. Etiquetas (tags) y versiones de lanzamiento**

6. Despliegue y mantenimiento de la documentación

- 6.1. Generación automática de documentación con herramientas CI/CD**
- 6.2. Publicación de documentación en la web (GitHub Pages, Read the Docs)**
- 6.3. Control de versiones en la documentación**
- 6.4. Actualización y mantenimiento continuo**

7. Buenas prácticas y casos prácticos

- 7.1. Estructura recomendada de un repositorio de proyecto**
- 7.2. Convenciones de mensajes de commit**
- 7.3. Ejemplo práctico: creación, control de versiones y despliegue de documentación de un proyecto**

8. Conclusiones y resumen de la unidad

1. Introducción

1.1. Importancia de la documentación en proyectos de software

La documentación es un elemento fundamental en cualquier proyecto de desarrollo de software. Su objetivo es dejar constancia del trabajo realizado, facilitar la comprensión del sistema y permitir su mantenimiento a lo largo del tiempo.

Una buena documentación ayuda a que otros desarrolladores (o el propio autor en el futuro) puedan entender cómo funciona la aplicación, cómo se instala, cómo se usa y cómo se puede modificar o ampliar.

Entre los tipos más habituales de documentación encontramos:

- Documentación técnica: describe la arquitectura, el código fuente, las dependencias y las configuraciones.
- Documentación de usuario: explica cómo utilizar la aplicación desde el punto de vista del cliente o usuario final.
- Documentación de instalación y despliegue: detalla los pasos necesarios para poner la aplicación en funcionamiento en distintos entornos.

La falta de documentación puede provocar errores, pérdida de tiempo y dificultades para el mantenimiento o evolución del software.

1.2. Beneficios del control de versiones

Un sistema de control de versiones permite registrar los cambios realizados sobre los archivos de un proyecto a lo largo del tiempo. De esta forma, es posible consultar el historial de modificaciones, restaurar versiones anteriores y trabajar de forma colaborativa sin perder información.

Entre sus principales beneficios destacan:

- Historial de cambios: cada modificación queda registrada con la fecha, el autor y una descripción.
- Recuperación de versiones anteriores: si se introduce un error, se puede volver fácilmente a un estado previo del proyecto.
- Trabajo colaborativo: varios desarrolladores pueden trabajar sobre el mismo proyecto sin sobrescribir el trabajo de otros.
- Trazabilidad: se puede conocer qué cambios se realizaron, cuándo y por qué.

El control de versiones es, por tanto, una herramienta esencial para el desarrollo profesional de software y la gestión eficiente de proyectos.

1.3. Flujo de trabajo colaborativo y trazabilidad

En los proyectos donde participa más de una persona, es necesario establecer un flujo de trabajo claro que defina cómo se realizan y gestionan los cambios. Los sistemas de control de versiones, como Git, facilitan esta organización mediante el uso de ramas (branches), fusiones (merges) y repositorios remotos.

Un flujo de trabajo colaborativo típico incluye:

1. Cada desarrollador trabaja en su propia rama para implementar nuevas funciones o corregir errores.
2. Los cambios se revisan y fusionan con la rama principal una vez verificados.
3. El repositorio remoto (por ejemplo, en GitHub o GitLab) se sincroniza para mantener una copia centralizada y actualizada del proyecto.

La trazabilidad consiste en poder seguir el rastro de cada cambio: saber quién lo hizo, cuándo y con qué propósito. Esto aporta transparencia y control, lo que resulta esencial en equipos de desarrollo, auditorías de calidad y mantenimiento de software.

2. Documentación en el desarrollo de aplicaciones web

La documentación es una parte esencial del ciclo de vida de cualquier aplicación web. No solo sirve para describir el funcionamiento del software, sino también para garantizar su mantenimiento, evolución y correcta utilización.

En un entorno profesional, la documentación debe acompañar al proyecto desde su fase inicial hasta su despliegue y posterior actualización. Una aplicación web sin documentación se vuelve difícil de mantener, entender o escalar, especialmente cuando intervienen varios desarrolladores o cuando el proyecto se extiende en el tiempo.

2.1. Tipos de documentación

Existen distintos tipos de documentación, según su propósito y público objetivo. En un proyecto completo, es habitual combinar varios de ellos para cubrir todas las necesidades.

a) Documentación técnica

Dirigida a los desarrolladores y administradores del sistema. Describe la estructura interna de la aplicación, su arquitectura, las tecnologías empleadas y las dependencias necesarias.

Incluye aspectos como:

- Estructura de directorios y archivos del proyecto.
- Descripción de clases, funciones, módulos y API.
- Requisitos del entorno de ejecución (versiones de frameworks, librerías, bases de datos, etc.).

- Diagramas UML, diagramas de flujo y esquemas de arquitectura.
- Procedimientos de compilación o despliegue.

Esta documentación es esencial para el mantenimiento, depuración y evolución del software.

b) Documentación de usuario

Explica cómo utilizar la aplicación desde el punto de vista del usuario final o cliente. Suele estar escrita con un lenguaje más accesible y puede incluir:

- Guías paso a paso.
- Capturas de pantalla o vídeos explicativos.
- Ejemplos de uso.
- Resolución de problemas frecuentes (FAQ).

Su objetivo es mejorar la experiencia del usuario y reducir la necesidad de soporte técnico.

c) Documentación de instalación y despliegue

Detalla los pasos necesarios para poner en marcha la aplicación web, tanto en entornos de desarrollo como de producción.

Incluye:

- Requisitos del sistema (hardware, software, dependencias).
- Configuración del servidor o contenedor.
- Variables de entorno.
- Comandos para la instalación o despliegue automatizado (scripts, Docker, CI/CD).

Esta documentación es especialmente útil para equipos de DevOps o administradores de sistemas.

d) Documentación de mantenimiento y soporte

Describe procedimientos para actualizar el software, corregir errores y realizar copias de seguridad. También puede incluir el registro de incidencias, versiones lanzadas y cambios realizados.

e) Documentación del proyecto

Incluye información general sobre el objetivo del proyecto, los roles de los miembros del equipo, el calendario de desarrollo y las decisiones técnicas tomadas.

Un ejemplo común es el archivo **README.md**, presente en la mayoría de repositorios.

2.2. Buenas prácticas para la redacción de documentación

La calidad de la documentación no depende solo de su extensión, sino de su **claridad, estructura y mantenimiento**. Algunas buenas prácticas recomendadas son:

1. Redactar con un lenguaje claro y preciso.

Evitar tecnicismos innecesarios o explicaciones vagas. Cada término debe estar bien definido y ser coherente con el resto del documento.

2. Actualizar la documentación junto con el código.

Una documentación desactualizada puede ser peor que no tener ninguna. Debe revisarse cada vez que se introduzcan cambios relevantes en el proyecto.

3. Usar un formato estándar.

Markdown es una buena opción por su simplicidad y compatibilidad con múltiples plataformas. También se pueden emplear formatos como reStructuredText o HTML.

4. Estructurar la información jerárquicamente.

Dividir el contenido en secciones, con títulos y subtítulos, para que sea fácil de navegar y localizar información concreta.

5. Incluir ejemplos y fragmentos de código.

Los ejemplos prácticos ayudan a comprender conceptos complejos más rápidamente que largas explicaciones teóricas.

6. Incorporar control de versiones.

Mantener la documentación en el mismo repositorio que el código permite controlar su evolución y asegurar la coherencia entre ambos.

7. Usar plantillas o guías de estilo.

Facilitan la uniformidad del formato y reducen el esfuerzo de redacción en equipos grandes.

8. Añadir metadatos y referencias.

Indicar el autor, la fecha de actualización y enlaces a recursos externos o documentación relacionada.

2.3. Herramientas para la generación de documentación

Existen numerosas herramientas que ayudan a crear, mantener y publicar documentación técnica de manera eficiente. Entre las más utilizadas destacan:

a) Markdown

Un lenguaje de marcado ligero que permite escribir texto con formato simple y legible.

Ventajas:

- Fácil de aprender y escribir.
- Compatible con plataformas como GitHub, GitLab o Bitbucket.
- Puede convertirse a HTML, PDF u otros formatos automáticamente.
Suele usarse para archivos como **README.md**, **CHANGELOG.md** o **CONTRIBUTING.md**.

b) Sphinx

Herramienta muy utilizada en proyectos Python, aunque adaptable a otros lenguajes. Permite generar documentación profesional a partir de archivos en **reStructuredText** o **Markdown**, exportando a múltiples formatos (HTML, PDF, ePub...).

Admite extensiones, generación automática de índices y vinculación con el código fuente.

c) Doxygen

Especialmente útil para proyectos en C, C++, Java o Python.

Genera documentación de forma automática a partir de los comentarios incluidos en el código, siguiendo un formato específico.

Produce resultados en HTML, LaTeX o RTF, entre otros, y puede integrarse en pipelines de CI/CD.

d) JSDoc

Herramienta para documentar proyectos en **JavaScript** o **TypeScript**.

Permite describir funciones, clases, parámetros y retornos mediante comentarios estructurados en el código.

e) MkDocs

Permite generar sitios web de documentación escritos en Markdown. Es sencilla, rápida y muy utilizada para documentar proyectos en GitHub, combinada con **GitHub Pages**.

f) Read the Docs

Plataforma que aloja y genera documentación automáticamente desde un repositorio. Integra herramientas como Sphinx o MkDocs y mantiene versiones sincronizadas con el código.

2.4. Integración de la documentación en el ciclo de vida del software

La documentación no debe verse como una tarea aislada o posterior al desarrollo, sino como parte integral del ciclo de vida del software.

Cada fase del proceso de desarrollo debe ir acompañada de su correspondiente documentación:

Fase del ciclo	Tipo de documentación asociada
Análisis	Requisitos funcionales y no funcionales, casos de uso.
Diseño	Diagramas UML, estructura de bases de datos, arquitectura del sistema.
Implementación	Comentarios en el código, documentación técnica y API.
Pruebas	Planes de prueba, resultados, informes de errores.
Despliegue	Guías de instalación, configuración y automatización.
Mantenimiento	Registro de versiones, incidencias y mejoras.

Además, gracias a las herramientas de **integración continua (CI/CD)**, es posible automatizar la generación y publicación de la documentación cada vez que se actualiza el código.

De esta manera, los equipos pueden asegurar que la documentación esté siempre al día y accesible desde el repositorio o desde un sitio web dedicado.

La **documentación viva** (living documentation) es una tendencia actual que busca mantener sincronizados el código, las pruebas y la documentación, reduciendo el riesgo de inconsistencias y mejorando la calidad general del software.

3. Sistemas de control de versiones

Los **sistemas de control de versiones (VCS, Version Control Systems)** son herramientas que permiten registrar y gestionar los cambios realizados sobre los archivos de un proyecto a lo largo del tiempo.

Su función principal es **guardar el historial de modificaciones, coordinar el trabajo entre varios desarrolladores y mantener la integridad del código fuente**.

El uso de un sistema de control de versiones es una práctica estándar en el desarrollo profesional de software. Gracias a estas herramientas, los equipos pueden trabajar de forma colaborativa sin perder información, evitando sobrescrituras o pérdidas de código.

3.1. Conceptos básicos

Para entender cómo funcionan los sistemas de control de versiones, es importante conocer algunos conceptos fundamentales:

a) Repositorio

Es el espacio donde se almacena todo el proyecto junto con su historial de versiones.

Puede ser **local** (en el equipo del desarrollador) o **remoto** (en un servidor o plataforma como GitHub o GitLab).

El repositorio guarda no solo los archivos actuales, sino también todas las versiones anteriores, los metadatos de los cambios y la información sobre los colaboradores.

b) Commit

Cada **commit** representa una “fotografía” del proyecto en un momento concreto.

Incluye los cambios realizados, el autor, la fecha y un mensaje descriptivo.

Los commits permiten reconstruir el estado del proyecto en cualquier punto del tiempo y forman la base del historial del repositorio.

Ejemplo: git commit -m "Corrige error en la validación del formulario de login"

c) Rama (Branch)

Una **rama** es una línea independiente de desarrollo dentro del proyecto.

Permite trabajar en nuevas funcionalidades, correcciones o experimentos sin afectar la versión principal (normalmente llamada **main** o **master**).

Cuando el trabajo en una rama está listo, puede **fusionarse** con la rama principal para incorporar los cambios.

Ejemplo: git checkout -b feature/login crea una nueva rama para desarrollar la funcionalidad de inicio de sesión.

d) Fusión (Merge)

El proceso de **fusión** combina los cambios de una rama con otra.

Generalmente se fusiona una rama de desarrollo o característica con la rama principal del proyecto.

Durante la fusión, el sistema analiza las diferencias entre ramas y aplica los cambios de manera automática, siempre que no haya conflictos.

e) Conflicto

Un **conflicto** ocurre cuando dos cambios afectan la misma parte de un archivo y el sistema no puede decidir cuál conservar.

En ese caso, el desarrollador debe resolver el conflicto manualmente, eligiendo qué versión mantener o integrando ambas.

Ejemplo típico: dos programadores editan la misma línea de un archivo al mismo tiempo.

Estos conceptos permiten comprender el flujo de trabajo básico en herramientas como **Git**, donde los desarrolladores pueden crear, modificar y combinar versiones de forma segura y organizada.

3.2. Tipos de sistemas de control de versiones

Los sistemas de control de versiones pueden clasificarse principalmente en dos categorías: **centralizados** y **distribuidos**.

a) Sistemas de control de versiones centralizados (CVCS)

En este modelo, existe un **servidor central** que almacena el repositorio principal, y los desarrolladores trabajan con copias locales de los archivos.

Cada vez que un usuario realiza un cambio, debe conectarse al servidor para **obtener la última versión** o **subir sus modificaciones**.

Ejemplos:

- CVS (Concurrent Versions System)
- Subversion (SVN)
- Perforce

Ventajas:

- Control centralizado del proyecto.
- Sencillez en la administración.
- Adecuado para entornos donde se requiere supervisión estricta.

Desventajas:

- Depende de la conexión con el servidor central.
- Si el servidor falla, el equipo no puede trabajar ni acceder al historial.
- Menor flexibilidad para trabajar sin conexión.

b) Sistemas de control de versiones distribuidos (DVCS)

En los sistemas distribuidos, **cada desarrollador tiene una copia completa del repositorio**, incluyendo todo el historial de cambios.

Esto permite trabajar de forma **independiente y sin conexión**, realizando commits localmente y sincronizando con el repositorio remoto cuando sea necesario.

Ejemplos:

- Git
- Mercurial
- Bazaar

Ventajas:

- No depende de un servidor central: se puede trabajar sin conexión.
- Mayor velocidad en operaciones locales (commit, branch, merge).
- Facilidad para crear ramas y fusionarlas.
- Copias de seguridad naturales: cada clon es un respaldo completo.

Desventajas:

- Requiere más espacio en disco al duplicar el historial.
- Puede ser más complejo para usuarios principiantes.

En la actualidad, **Git** es el sistema de control de versiones más utilizado gracias a su potencia, flexibilidad y a las plataformas que lo acompañan (GitHub, GitLab, Bitbucket, etc.).

3.3. Ventajas del uso de sistemas de control de versiones

Adoptar un sistema de control de versiones ofrece numerosas ventajas tanto a nivel técnico como organizativo. Entre las más importantes destacan:

a) Historial completo de cambios

Cada modificación queda registrada con información detallada: autor, fecha, descripción y contenido modificado.

Esto permite consultar el origen de un cambio o identificar cuándo se introdujo un error.

b) Recuperación de versiones anteriores

Es posible restaurar el estado del proyecto a cualquier versión anterior, lo que resulta útil ante errores, pérdidas de datos o regresiones.

c) Trabajo colaborativo eficiente

Varios desarrolladores pueden trabajar simultáneamente sobre el mismo proyecto, incluso en distintas partes del código, sin interferirse entre sí.

Las herramientas de fusión permiten integrar fácilmente los cambios de diferentes colaboradores.

d) Control de ramas y desarrollo paralelo

Las ramas posibilitan desarrollar nuevas funcionalidades, realizar pruebas o corregir errores sin afectar la versión estable.

Esto facilita la gestión de distintas versiones del software (por ejemplo, desarrollo, pruebas y producción).

e) Trazabilidad y auditoría

Permite saber quién hizo cada cambio, cuándo y con qué propósito. Esta trazabilidad es fundamental para auditorías, mantenimiento y control de calidad.

f) Automatización y despliegue continuo

Integrado con herramientas de integración continua (CI/CD), el control de versiones permite automatizar procesos como pruebas, compilación y despliegue de aplicaciones web.

g) Copias de seguridad y seguridad del código

Cada clon del repositorio actúa como copia de seguridad. Además, los repositorios pueden estar protegidos mediante permisos de acceso y autenticación.

h) Mejora del trabajo en equipo

Favorece la comunicación y coordinación entre los miembros del equipo, ya que todos pueden visualizar el progreso, las tareas pendientes y los cambios en curso.

En resumen, un sistema de control de versiones no solo organiza y protege el código, sino que también **optimiza el flujo de trabajo, mejora la productividad y garantiza la calidad del software**.

Su uso es indispensable en cualquier entorno de desarrollo profesional, especialmente en proyectos colaborativos o de larga duración.

4. Instalación y configuración de un sistema de control de versiones

En la actualidad, el sistema de control de versiones más utilizado es **Git**, creado por Linus Torvalds en 2005. Git es un sistema **distribuido**, rápido, seguro y gratuito, ampliamente adoptado por desarrolladores y empresas. A continuación, se detallan los pasos necesarios para su instalación, configuración inicial y conexión con plataformas de alojamiento remoto.

4.1. Instalación de Git en diferentes sistemas operativos

Git puede instalarse en cualquier sistema operativo moderno. El proceso varía ligeramente según la plataforma:

a) Instalación en Windows

1. Acceder al sitio oficial: <https://git-scm.com/downloads>.
2. Descargar el instalador para Windows y ejecutar el archivo .exe.
3. Durante la instalación, se recomienda mantener las opciones por defecto, aunque se pueden ajustar:
 - **Editor por defecto:** elegir entre *Vim*, *Notepad++*, *Visual Studio Code*, etc.
 - **Ruta del PATH:** seleccionar “Git from the command line and also from 3rd-party software”.
 - **Conversión de finales de línea (EOL):** elegir “Checkout Windows-style, commit Unix-style line endings”.
4. Una vez instalado, abrir **Git Bash** o **PowerShell** y comprobar la instalación con:

```
git --version
```

b) Instalación en Linux

En la mayoría de distribuciones, Git está disponible desde los repositorios oficiales.

- En **Debian / Ubuntu:**

```
sudo apt update  
sudo apt install git
```

- En **Fedora / Red Hat:**

```
sudo dnf install git
```

- En **Arch Linux:**

```
sudo pacman -S git
```

Verificar la instalación:

```
git --version
```

c) Instalación en macOS

En macOS, Git puede instalarse de varias formas:

- **Usando Homebrew:**

```
brew install git
```

- **Desde Xcode Command Line Tools:**

```
xcode-select --install
```

- O descargando el instalador desde la web oficial.

Comprobar la instalación con:

```
git --version
```

4.2. Configuración inicial de Git (nombre, correo, editor, SSH)

Una vez instalado Git, es necesario realizar una **configuración inicial** que se almacenará en el archivo `.gitconfig` del usuario.

Esta configuración identifica al autor de los cambios y define parámetros básicos del entorno.

a) Configuración del nombre y correo electrónico

Estos datos aparecerán en cada commit realizado:

```
git config --global user.name "Iván Jiménez García"
```

```
git config --global user.email "ivan@example.com"
```

El modificador `--global` aplica la configuración a todos los repositorios del usuario.

Si se desea configurar un repositorio concreto, se omite este modificador.

b) Configuración del editor por defecto

Git usa un editor de texto para escribir mensajes de commit o resolver conflictos.

Puede cambiarse por el editor preferido, por ejemplo:

```
git config --global core.editor "code --wait"
```

(para Visual Studio Code)

O bien:

```
git config --global core.editor "nano"
```

c) Configuración de colores y formato de salida

Para mejorar la legibilidad de los comandos en la terminal:

```
git config --global color.ui auto
```

d) Configuración de autenticación SSH

La conexión mediante **SSH (Secure Shell)** permite interactuar con plataformas remotas sin introducir usuario y contraseña cada vez.

Pasos básicos:

1. Generar un par de claves SSH:

```
ssh-keygen -t ed25519 -C "ivan@example.com"
```

(usar RSA si el sistema no admite ED25519)

2. Iniciar el agente SSH y añadir la clave privada:

```
eval "$(ssh-agent -s)"
```

```
ssh-add ~/.ssh/id_ed25519
```

3. Copiar la clave pública y añadirla a la cuenta en GitHub, GitLab o Bitbucket:

```
cat ~/.ssh/id_ed25519.pub
```

Luego pegar el contenido en el apartado **SSH Keys** de la plataforma.

4. Probar la conexión:

```
ssh -T git@github.com
```

Si la configuración es correcta, aparecerá un mensaje de bienvenida.

4.3. Creación de un repositorio local y remoto

Git permite trabajar tanto con repositorios **locales** (en el propio equipo) como **remotos** (en servidores o servicios online). Un flujo de trabajo típico comienza creando el repositorio local y posteriormente vinculándolo a un repositorio remoto.

a) Crear un repositorio local

1. Acceder al directorio del proyecto:

```
cd ~/Documentos/mi_proyecto
```

2. Inicializar el repositorio:

```
git init
```

Esto crea una carpeta oculta llamada `.git` que contiene toda la información del control de versiones.

3. Añadir archivos al control de versiones:

```
git add .
```

4. Registrar los cambios:

```
git commit -m "Versión inicial del proyecto"
```

b) Crear un repositorio remoto

En plataformas como GitHub o GitLab se puede crear un nuevo repositorio desde la interfaz web. Al hacerlo, la plataforma generará una URL del tipo:

```
git@github.com:usuario/mi_proyecto.git
```

o (en HTTPS):

```
https://github.com/usuario/mi_proyecto.git
```

c) Vincular el repositorio local con el remoto

Una vez creado el remoto, se puede conectar con:

```
git remote add origin git@github.com:usuario/mi_proyecto.git
```

Comprobar la vinculación:

```
git remote -v
```

Finalmente, subir los archivos iniciales:

```
git push -u origin main
```

4.4. Conexión con plataformas de alojamiento (GitHub, GitLab, Bitbucket)

Los repositorios remotos suelen alojarse en servicios especializados que ofrecen funcionalidades adicionales, como control de acceso, seguimiento de incidencias o integración continua. Las más utilizadas son:

a) GitHub

- Plataforma más popular del ecosistema Git.
- Permite alojamiento gratuito de repositorios públicos y privados.
- Ofrece integración con **GitHub Actions** para CI/CD, gestión de incidencias, wikis y páginas web mediante **GitHub Pages**.
- Admite autenticación mediante SSH, tokens personales o claves GPG para firmas de commits.

Ejemplo de flujo básico:

1. Crear el repositorio en GitHub.
2. Conectar el remoto con:

```
git remote add origin git@github.com:usuario/mi_proyecto.git
```

3. Subir los cambios:

```
git push origin main
```

b) GitLab

- Similar a GitHub, pero con más control sobre la infraestructura.
- Permite desplegar una **instancia propia** de GitLab en servidores locales (ideal para empresas o centros educativos).
- Integra un potente sistema de **CI/CD**, control de permisos, gestión de tareas y wikis internas.

Ejemplo de conexión:

```
git remote add origin git@gitlab.com:usuario/mi_proyecto.git
```

c) Bitbucket

- Plataforma desarrollada por Atlassian.
- Soporta **Git** y **Mercurial** (aunque este último ya en desuso).
- Destaca por su integración con herramientas de gestión como **Jira** y **Confluence**.
- Ofrece planes gratuitos con repositorios privados.

Ejemplo de conexión:

```
git remote add origin git@bitbucket.org:usuario/mi_proyecto.git
```

Conclusión del apartado

La instalación y configuración de Git constituyen el primer paso para gestionar proyectos de desarrollo de forma profesional. Una correcta configuración garantiza que cada cambio quede identificado, que el trabajo en equipo sea fluido y que el código esté siempre respaldado y accesible. Con el repositorio sincronizado con plataformas como GitHub o GitLab, se habilitan además funcionalidades clave para el desarrollo moderno: integración continua, revisión de código, control de versiones y despliegue automatizado.

5. Uso básico y avanzado de Git

Una vez instalado y configurado Git, es fundamental dominar su uso tanto a nivel básico como en tareas avanzadas. Git permite registrar cambios, gestionar ramas, resolver conflictos y coordinar el trabajo entre distintos desarrolladores de forma eficiente.

5.1. Comandos esenciales: init, add, commit, status, log

git init

Inicializa un nuevo repositorio en el directorio actual, creando la carpeta oculta `.git` donde se guardará toda la información del historial.

```
git init
```

git add

Añade archivos al área de preparación (*staging area*), indicando que se incluirán en el próximo commit.

```
git add archivo.txt    # Añade un archivo
```

```
git add .              # Añade todos los cambios del directorio
```

git commit

Guarda los cambios añadidos con un mensaje descriptivo.

```
git commit -m "Descripción del cambio"
```

git status

Muestra el estado actual del repositorio: archivos modificados, añadidos o pendientes de confirmar.

```
git status
```

git log

Muestra el historial de commits con detalles como autor, fecha y mensaje.

```
git log                # Vista completa
```

```
git log --oneline      # Vista resumida
```

5.2. Trabajo con ramas: branch, checkout, merge, rebase

Git permite trabajar con **ramas (branches)** para desarrollar nuevas funcionalidades sin afectar el código principal.

git branch

Lista las ramas existentes o crea una nueva.

```
git branch          # Muestra las ramas
```

```
git branch nueva-rama  # Crea una nueva rama
```

git checkout

Permite cambiar de rama o restaurar archivos.

```
git checkout nueva-rama  # Cambia a la rama indicada
```

```
git checkout main      # Regresa a la rama principal
```

git merge

Combina los cambios de una rama con otra. Generalmente se usa para fusionar una rama de desarrollo con main.

```
git checkout main
```

```
git merge nueva-rama
```

git rebase

Reaplica commits de una rama sobre otra, generando un historial más lineal.

```
git checkout nueva-rama
```

```
git rebase main
```

⚠ Debe usarse con precaución en repositorios compartidos, ya que reescribe el historial.

5.3. Resolución de conflictos

Un **conflicto** ocurre cuando dos ramas modifican la misma parte de un archivo y Git no puede decidir qué versión conservar.

Pasos para resolver conflictos:

1. Git marca los archivos en conflicto con secciones delimitadas por <<<<<<<, ===== y >>>>>>>.
2. Editar manualmente los archivos para conservar la versión deseada.
3. Añadir los archivos corregidos al *staging area*:

```
git add archivo_conflictivo.txt
```
4. Finalizar la fusión o rebase:

```
git merge --continue
```

```
# o
```

```
git rebase --continue
```

Se recomienda usar herramientas visuales de comparación y fusión, como las integradas en VS Code, GitKraken o Sourcetree.

5.4. Trabajo colaborativo con repositorios remotos: clone, fetch, pull, push

Git permite colaborar a través de repositorios remotos alojados en plataformas como GitHub o GitLab.

git clone

Copia un repositorio remoto en local.

```
git clone git@github.com:usuario/proyecto.git
```

git fetch

Descarga los cambios del repositorio remoto sin aplicarlos todavía.

```
git fetch origin
```

git pull

Descarga los cambios remotos y los fusiona automáticamente con la rama local.

```
git pull origin main
```

git push

Envía los commits locales al repositorio remoto.

```
git push origin main
```

En equipos de desarrollo, es buena práctica realizar un git pull antes de hacer push, para evitar conflictos y mantener la sincronización con el servidor remoto.

5.5. Etiquetas (tags) y versiones de lanzamiento

Las **etiquetas (tags)** permiten marcar puntos específicos en la historia del proyecto, normalmente utilizados para identificar versiones estables o lanzamientos.

Crear una etiqueta ligera:

```
git tag v1.0
```

Crear una etiqueta anotada (con mensaje):

```
git tag -a v1.0 -m "Versión 1.0 estable"
```

Listar etiquetas existentes:

```
git tag
```

Compartir etiquetas con el repositorio remoto:

```
git push origin --tags
```

Las etiquetas son muy útiles para versionar el software, generar paquetes de distribución o marcar hitos importantes del desarrollo.

Conclusión

Dominar los comandos básicos y avanzados de Git permite trabajar de forma ordenada, eficiente y colaborativa. Desde la creación de ramas y commits hasta la resolución de conflictos y la gestión de versiones, Git se convierte en una herramienta esencial en cualquier entorno de desarrollo profesional.

6. Despliegue y mantenimiento de la documentación

La documentación técnica no debe considerarse un elemento estático del proyecto, sino una parte viva del ciclo de desarrollo. Su despliegue y mantenimiento permiten asegurar que siempre esté actualizada, accesible y alineada con el estado real del código. Hoy en día, gracias a las herramientas de **integración continua (CI/CD)** y las plataformas de publicación en línea, este proceso puede automatizarse casi por completo.

6.1. Generación automática de documentación con herramientas CI/CD

Las **herramientas de Integración y Despliegue Continuo (CI/CD)** permiten automatizar la generación y actualización de la documentación cada vez que se realizan cambios en el repositorio.

Flujo general:

1. El desarrollador actualiza el código y realiza un *commit*.
2. El sistema CI/CD detecta los cambios y ejecuta un pipeline automatizado.
3. En una de las etapas del pipeline, se genera la documentación de forma automática a partir de los comentarios del código o de archivos fuente (Markdown, reStructuredText, etc.).
4. Finalmente, la documentación se publica de manera automática en un servidor o página web.

Ejemplo de herramientas y plataformas CI/CD:

- **GitHub Actions:** permite definir workflows en archivos YAML que automatizan tareas como la generación de documentación con **Sphinx** o **Doxygen**.
- **GitLab CI/CD:** incluye pipelines configurables para compilar, probar y desplegar documentación.
- **Jenkins, CircleCI, Travis CI:** también pueden integrarse para generar y desplegar documentación.

Ejemplo básico de flujo CI/CD con GitHub Actions:

name: Build Docs

on:

push:

branches: ["main"]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Instalar dependencias

run: pip install sphinx

- name: Generar documentación

run: sphinx-build -b html docs/ build/

- name: Desplegar en GitHub Pages

uses: peaceiris/actions-gh-pages@v3

with:

github_token: \${{ secrets.GITHUB_TOKEN }}

publish_dir: ./build

Este flujo genera la documentación automáticamente cada vez que se actualiza la rama principal y la publica en GitHub Pages.

6.2. Publicación de documentación en la web (GitHub Pages, Read the Docs)

Publicar la documentación en la web permite que el equipo de desarrollo, los usuarios y los colaboradores accedan fácilmente a la información actualizada desde cualquier lugar.

a) GitHub Pages

- Servicio gratuito de GitHub que permite alojar sitios web directamente desde un repositorio.
- Ideal para publicar documentación generada con **MkDocs**, **Jekyll** o **Sphinx**.
- Pasos básicos:
 1. Crear una carpeta docs/ o una rama gh-pages en el repositorio.
 2. Configurar GitHub Pages desde la pestaña *Settings* → *Pages*.
 3. El sistema publicará automáticamente la documentación en una URL del tipo:

<https://usuario.github.io/nombre-del-proyecto/>

b) Read the Docs

- Plataforma orientada a la documentación técnica.
- Se integra directamente con repositorios de GitHub o GitLab.
- Cada vez que se actualiza el repositorio, Read the Docs reconstruye y publica automáticamente la documentación.
- Es compatible con Sphinx, MkDocs y otros generadores.
- Permite manejar múltiples versiones de la documentación (por ejemplo, v1.0, v2.0, latest).

c) Otras alternativas

- **GitLab Pages**, similar a GitHub Pages pero integrado con el ecosistema de GitLab.
- **Netlify** o **Vercel**, plataformas de despliegue continuo con soporte para sitios estáticos.

6.3. Control de versiones en la documentación

La documentación debe versionarse igual que el código fuente, para reflejar fielmente el estado de cada versión del software.

Buenas prácticas:

- Al mantener distintas ramas de desarrollo (main, dev, release), cada una puede tener su propia documentación actualizada.
- Utilizar etiquetas (*tags*) para generar documentación asociada a una versión concreta del producto.
- En plataformas como **Read the Docs**, es posible configurar múltiples versiones visibles para el usuario (por ejemplo: “versión 1.0”, “versión 2.1”, “última”).
- Evitar modificar la documentación directamente en la rama principal sin pasar por revisiones o *pull requests*, garantizando así su trazabilidad y coherencia con el código.

Ejemplo de flujo:

1. Se lanza la versión v2.0 del software.
2. Se etiqueta el código y la documentación con `git tag v2.0`.
3. El sistema CI/CD publica automáticamente la documentación de esa versión bajo `/docs/v2.0/`.

6.4. Actualización y mantenimiento continuo

La documentación, como el código, debe evolucionar con el proyecto. Un documento desactualizado puede generar confusión, errores de uso o incluso fallos en la implementación.

Recomendaciones clave para el mantenimiento:

- **Actualizar junto al código:** cualquier cambio funcional debe reflejarse en la documentación correspondiente.
- **Usar revisiones por pares (code review):** incluir la revisión de la documentación en el flujo de validación del proyecto.
- **Automatizar comprobaciones:** scripts o pipelines pueden verificar que los archivos de documentación estén actualizados y sin errores de compilación.
- **Monitorear enlaces y recursos:** utilizar herramientas automáticas (como *linkcheck* de Sphinx) para detectar enlaces rotos o recursos inaccesibles.
- **Establecer responsables:** designar roles dentro del equipo encargados del mantenimiento documental.
- **Registrar cambios (CHANGELOG):** mantener un historial claro de modificaciones tanto en el código como en la documentación.

Conclusión

El despliegue y mantenimiento de la documentación forman parte esencial del ciclo de vida del software. Mediante la automatización con CI/CD y la publicación en plataformas accesibles, los equipos garantizan que la información técnica esté siempre actualizada, versionada y disponible para todos los colaboradores. Una documentación bien gestionada mejora la calidad del proyecto, facilita la colaboración y asegura la sostenibilidad del desarrollo a largo plazo.

7. Buenas prácticas y casos prácticos

La calidad de un proyecto de software no solo depende del código, sino también de la organización del repositorio, la claridad de los mensajes de commit y la gestión coherente de su documentación. Aplicar buenas prácticas garantiza mantenibilidad, colaboración eficiente y trazabilidad completa del desarrollo.

7.1. Estructura recomendada de un repositorio de proyecto

Un repositorio bien estructurado facilita la comprensión del proyecto y su mantenimiento por parte de cualquier desarrollador o equipo. Aunque puede variar según el tipo de aplicación, una estructura estándar puede ser la siguiente:

/mi_proyecto

|

└─ /src/ # Código fuente principal

└─ /docs/ # Documentación técnica y de usuario

└─ /tests/ # Pruebas unitarias o de integración
└─ /assets/ # Recursos estáticos (imágenes, CSS, scripts, etc.)
└─ README.md # Descripción general del proyecto
└─ LICENSE # Licencia del software
└─ .gitignore # Archivos o carpetas que Git debe ignorar
└─ requirements.txt # Dependencias (en proyectos Python, por ejemplo)
└─ setup.cfg / package.json # Configuración del proyecto

Recomendaciones adicionales:

- Mantener el archivo README.md siempre actualizado y claro.
- Usar CHANGELOG.md para registrar los cambios importantes entre versiones.
- Ignorar con .gitignore archivos temporales, binarios o de entorno local.
- Organizar la documentación técnica dentro de /docs/ para integrarla fácilmente con Sphinx, MkDocs o similar.

7.2. Convenciones de mensajes de *commit*

Los mensajes de *commit* deben ser **claros, breves y descriptivos**, ya que forman parte del historial del proyecto y facilitan su comprensión. Un buen mensaje explica **qué se ha hecho y por qué**, sin necesidad de consultar el código.

Estructura recomendada:

<tipo>(<área>): <mensaje breve>

Ejemplos de tipos comunes:

- feat: → nueva funcionalidad
- fix: → corrección de errores
- docs: → cambios en documentación
- style: → formato o estilo del código
- refactor: → reestructuración sin cambio funcional
- test: → adición o modificación de pruebas
- chore: → tareas de mantenimiento

Ejemplo de commits bien redactados:

feat(auth): añadir validación de tokens JWT

fix(api): corregido error al parsear respuesta JSON

docs(readme): actualizar instrucciones de instalación

Estas convenciones (inspiradas en *Conventional Commits*) facilitan la generación automática de changelogs y la integración con sistemas CI/CD.

7.3. Ejemplo práctico: creación, control de versiones y despliegue de documentación de un proyecto

A continuación se describe un flujo de trabajo realista que integra todos los conceptos estudiados:

1. Inicialización del proyecto:

Se crea el directorio y se inicializa el repositorio Git:

```
mkdir mi_proyecto && cd mi_proyecto  
git init
```

2. Estructura base y documentación inicial:

Se añaden carpetas /src, /docs y un archivo README.md.

```
echo "# Proyecto Ejemplo" > README.md  
mkdir src docs  
git add .  
git commit -m "feat: estructura inicial del proyecto"
```

3. Creación del repositorio remoto y conexión:

Se crea un repositorio en GitHub y se enlaza con el local:

```
git remote add origin git@github.com:usuario/mi_proyecto.git  
git push -u origin main
```

4. Generación de documentación automática:

Se instala y configura Sphinx o MkDocs en la carpeta /docs/.
Ejemplo con MkDocs:

```
pip install mkdocs  
mkdocs new docs  
mkdocs serve
```

Luego se versiona la documentación:

```
git add docs/  
git commit -m "docs: añadir documentación inicial con MkDocs"
```

5. Despliegue automatizado de la documentación:

Se configura GitHub Pages o un flujo CI/CD para publicar la documentación automáticamente al actualizar la rama principal.

Por ejemplo, con MkDocs:

```
mkdocs gh-deploy
```

La documentación quedará disponible en:

https://usuario.github.io/mi_proyecto/

6. Mantenimiento continuo:

Cada vez que se modifica el código o la documentación, se realizan nuevos commits con mensajes claros, se revisan los cambios mediante *pull requests* y se regeneran automáticamente las páginas actualizadas.

Este flujo asegura coherencia entre código, documentación y versiones, integrando buenas prácticas de desarrollo profesional.

8. Conclusiones y resumen de la unidad

La documentación y el control de versiones son pilares fundamentales en el desarrollo web moderno.

A través del uso de **Git** y de herramientas complementarias como **Sphinx**, **MkDocs**, **GitHub Pages** o **Read the Docs**, es posible mantener un entorno de trabajo profesional, organizado y automatizado.

Puntos clave de la unidad:

- La documentación debe integrarse desde el inicio del proyecto y mantenerse actualizada.
- Git permite registrar la evolución del código y la documentación, garantizando trazabilidad y trabajo colaborativo.
- Los sistemas CI/CD facilitan la generación y despliegue automáticos de la documentación.
- Seguir buenas prácticas (estructura del repositorio, mensajes de commit, versionado) mejora la claridad y calidad del desarrollo.
- La combinación de control de versiones y documentación automatizada refuerza la sostenibilidad y fiabilidad del proyecto a largo plazo.

En definitiva, dominar estas herramientas y metodologías prepara al desarrollador para trabajar en entornos profesionales, colaborativos y alineados con los estándares actuales del desarrollo de software.